

Programming 2

Theory exam (July 2019)

July 4, 2019



Instructions

- **Duration: 3 hours**
- The first exercise should be called `gestor.cc`. For the second exercise it is necessary to submit four files, called `Tweet.cc`, `Tweet.h`, `User.cc` and `User.h`. Add your ID and name in a comment at the beginning of all the source files.
- The exam will be submitted like an assignment, using the DLSI web server (<http://pracdlisi.dlsi.ua.es>), in the Programación 2 link. You can upload several versions, although only the last submission will be reviewed.
- In the Programming 2 webpage <http://www.dlsi.ua.es/assignaturas/p2> you can download some files to implement the exam exercises, and also the Reference from www.cplusplus.com.

Exercises

1. (4.5 points)

Our company has changed its management program and we need to migrate the customer data to the new format. The objective of this exercise is to develop a program that receives the data in the old format as input and converts it to the new format. The program should be able to perform two tasks: (i) read the original format in binary and transform it into an intermediate format in a text file and, (ii) from this text file, generate the target format in binary.

The data of the old program are stored in a binary file whose registers have the following structure:

```
char name[20];           // Customer name
char surname1[20];       // First Surname
char surname2[20];       // Second surname
char address[100];       // Address
int balance;             // Balance in the customer account
```

The program must read a binary file and save the data into a text file, where the information of each client will be in a different line with the following format:

```
name|surname1|surname2|address|balance
```

If any user of the binary file has a negative balance, the value NA (*not available*) will be stored in the text file instead of that negative balance. For example:

```
Pablo|Castro|Ortega|C/ Mota del Carmen, 36, Sevilla|350
Ana|Escobar|Soriano|C/ Alhambra, 22, Madrid|420
Luis|Esteban|Torres|C/ San Vicente, 33, Alicante|NA
```

From this text file, the program must be able to read the information and save it in another binary file with the format of the new program. The binary structs of the new program should have this format:

```
char fullName[75];       // Name and surnames
char address[75];        // Address
float balance;           // Balance in the customer account
```

The name and the two last names will be stored in `fullName`, separated by a blank space. The `address` will be stored just as it was in the old format, but bearing in mind that the new field has a smaller size than the original and the text must be cut to fit. The field `balance` contains the same information as in the original, although in this case it is a field of type `float` instead of `int`. If the balance read from the text file is `NA`, it will be assigned the value 0 in the new format.

The program will receive an input parameter to indicate what type of operation we want to perform with it:

- The option `-l` will allow reading a binary file in the old format and generate an intermediate text file with the format mentioned above
- The option `-s` will allow reading an intermediate text file and save the data into a binary file with the new format

In both cases, two more parameters will be received: first the name of the binary file and then the name of the text file. For example:

```
$ gestor -l datos_old.bin temp.txt
```

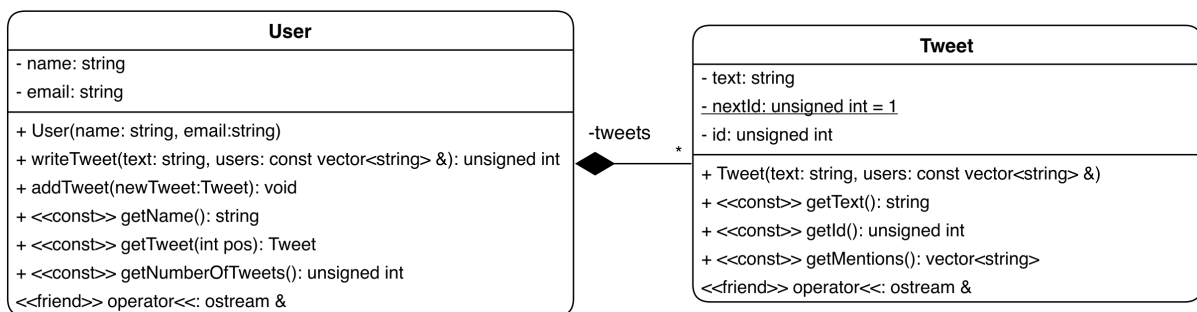
will read the customer data in the old format from `data_old.bin` and save it in the intermediate text file `temp.txt`, in the aforementioned format. On the other hand:

```
$ gestor -s datos_new.bin temp.txt
```

will read the data from the intermediate file `temp.txt` and generate a binary file `data_new.bin` with the new format.

The parameters will always be supplied in that order and it will not be necessary to check it: first the option, followed by the name of the binary file and followed by the name of the text file. However, you will need to verify that the number of parameters is correct and that the first of them is `-l` or `-s`. Otherwise, an error will be displayed and the program should finish.

2. (5.5 points)



We want to make a program to manage a simplified version of Twitter. This program would allow a user to write text messages, called *tweets* in the jargon of the application. As you can see in the diagram, this program will have two classes: **User** (users that send messages) and **Tweet** (the messages themselves).

Objects of class **Tweet** represent a message or *tweet*. In the file **Tweet.h** you must declare the following enum type, which contains the possible exceptions that may occur in this program:

```
enum Exception { exception_mention, exception_name, exception_email, exception_pos };
```

The text of a *tweet* can mention one or more users using the symbol `@` before its name. The name of a user contains only alphanumeric characters (letters and numbers).¹ For example, the *tweet* `are you sure it was me, @juan?` mentions the user with name `juan`.

¹You can use the function `isalnum` from the library `cctype`, which receives as only parameter a character and returns `true` if it is alphanumeric, and `false` otherwise.

The constructor of `Tweet` receives, in addition to the text of *tweet*, a vector of user names that is used to check the users mentioned in the text of that *tweet*: if it is mentioned in the text to a user not included in the vector, an exception of type `exception_mention` will be thrown. Each *tweet* will have a unique numeric identifier, starting with 1. The identifier to be used for a new *tweet* will be the value that the static attribute `nextId` has at that moment, which will be incremented in this constructor to be used in the next *tweet*.

The method `getMentions` returns a list with the names of the users mentioned in the text of the *tweet*. This list should not contain duplicates, so if a user is mentioned several times, only one will be included. Finally, the operator `<<` for an object of the class `Tweet` shows its identifier in brackets followed by a blank space and the text of *tweet*. For example:

```
[3] just saw @arya at the supermarket!
```

A `User` has a `name` and an `email` address. The constructor of the class `User` receives both values. First you must check that the user's name is correct, that is, that it only contains alphanumeric characters. Otherwise, it will throw the exception `exception_name`. Second, check that the email address is well formed. In case it is not, it will throw an exception of type `exception_email`. For simplicity, this exercise assumes that an email address is well formed if it starts with a sequence of one or more alphanumeric characters, followed by the character '@', and it is in turn followed by another sequence of alphanumeric characters that must contain at least one dot ('.'), which can not be in the first or last position of that sequence. For example, `pepe@.com` or `pepe@com.` would not be valid, because the '.' is at the beginning of the sequence in the first case and at the end of it in the second one.

The `writeTweet` method creates an object of the class `Tweet` and adds it to the *tweets* list of the user. In case the constructor of the class `Tweet` throws an exception, this method will capture it and show an error on the screen indicating that the *tweet* could not be created. If it was created correctly, this method should return its identifier and otherwise return 0. The `addTweet` method allows us to add a *tweet* to the list of messages sent by the user. We will assume that the *tweet* passed as a parameter has been created correctly, so there is no need to do any additional checking on this method.

The method `getTweet` returns the *tweet* that the position `pos` occupies in the list of *tweets* of the user. If the position is outside the range of the vector, the program will throw an exception of type `exception_pos`. The method `getNumberOfTweets` returns the number of *tweets* written by the user. Finally, the operator `<<` for an object of the class `User` will show the data of a user and their messages, following the format of the following example:

```
juan (johnny5@google.com)
[1] just saw @arya at the supermarket!
[2] @juan and @arya bought the same cookies :-)
[3] good afternoon, world
```

Note: No public methods can be added to the public part, but they can be private.

Given the next file `main.cc`, that can be compiled with `g++ Tweet.cc User.cc main.cc -o ej2`:

```
int main()
{
    vector<string> users;

    User juan("juan","john666@google.com");
    users.push_back(juan.getName());

    User arya("arya12","aryaPulova@hotmail.es");
    users.push_back(arya.getName());

    juan.writeTweet("just saw @arya12 at the supermarket!",users);
    arya.writeTweet("are you sure it was me, @juan?",users);
    Tweet t1("good morning, world",users);
    arya.addTweet(t1);
    Tweet t2("good afternoon, world",users);
    juan.addTweet(t2);

    cout << juan << endl;
    cout << arya << endl;
}
```

The program output should be:

```
john (john666@google.com)
[1] just saw @arya12 at the supermarket!
[4] good afternoon, world

arya12 (aryaPulova@hotmail.es)
[2] are you sure it was me, @juan?
[3] good morning, world
```